

Here's the detailed plan, based on Claude Code's current docs (memory, rules, subagents, skills, hooks, settings) and a full pass through your 13 project docs + the two repo screenshots.

The problem this plan has to solve first

Before designing folders, one thing matters more: **your docs contradict each other**, and Claude Code's own guidance is explicit that contradictory instructions get "picked arbitrarily." Concretely — README.md and onboarding.md both say core_cavity.py is *not implemented*; guide.md, working.md, understanding.md, and IMPLEMENTATION_STATUS.md all say it *is* implemented (face-classification only, no Boolean split). If I naively dump all 13 files into .claude/, Claude Code inherits that contradiction.

So step one is an **authority table** — one source of truth per topic, everything else treated as historical color that doesn't get fed into .claude/ verbatim:

Topic	Authoritative source	Why
Exact current code behavior	guide.md	reverse-engineered from source, cites line numbers, explicitly reconciles doc/code drift
What's implemented vs planned	IMPLEMENTATION_STATUS.md	self-declared "truth source"
Testing approach	testing_plan.md	matches actual tests/layout
Domain concepts (draft/undercut/parting line)	understanding.md Parts 1–2	most complete, cross-checked against code
Research paper fidelity	Engine.md + IMPLEMENTATION_STATUS.md's fidelity section	paper content + honest gap
Demo narration / what not to claim	DEMO_SCRIPT.md	has an explicit "claims to avoid" list
README.md, onboarding.md, understand.md, working.md	superseded / background only	overlap heavily with the above and contain the stale claims

Everything below is built on top of that table.

Research findings that shape the structure

From code.claude.com docs (memory, sub-agents, skills, hooks-guide, costs):

- **CLAUDE.md loads in full every single session** — target under 200 lines. Anthropic's #1 stated cost-reduction tip is literally "move instructions from CLAUDE.md to skills."

- **Custom commands have been merged into skills.** `.claude/commands/*.md` still works, but `.claude/skills/<name>/SKILL.md` is the modern mechanism — it adds supporting files, invocation control, and subagent execution, and only loads its body on demand (the description is the only thing always in context).
- **Subagents run in their own context window.** Anthropic's #2 stated tip: "delegate verbose operations to subagents" — running tests, processing logs. This project's `testing_plan.md` literally instructs `pytest tests/ -v --tb=short -s everywhere`, which is exactly the verbose-output case subagents exist for.
- **Hooks are deterministic, not persuasive** — they can preprocess noisy output *before* Claude ever sees it (e.g., grep a test run down to just FAIL/ERROR lines), and they can hard-block an action regardless of what the model decides. `CLAUDE.md/rules` only ever *persuade*.
- **`.claude/rules/*.md` can be path-scoped** (paths: frontmatter) — they only enter context when Claude actually opens a matching file. Perfect for this repo's clean module boundaries (geometry engine vs. API vs. frontend vs. tests).
- **Real auto-memory lives outside the repo**, at `~/.claude/projects/<project>/memory/`, and is per-machine. A repo-committed `.claude/memory/` (like your reference screenshot has) is a different, manual convention — useful, but I'll label it clearly so it's not confused with the built-in feature.

Proposed tree

```

Bosch/
├── CLAUDE.md                # root memory, ~150-180 lines
├── .claude/
│   ├── settings.json        # shared: permissions + hooks (checked in)
│   ├── settings.local.json  # personal overrides (gitignored)
│   └── rules/
│       ├── geometry-engine.md    # paths: backend/geometry/**,
backend/models/**
│       ├── api-layer.md          # paths: backend/api/**
│       ├── frontend.md          # paths: frontend/**
│       ├── testing.md           # paths: tests/**
│       ├── config-and-infra.md   # paths: config.yaml, docker-
compose.yml, environment.yml, Dockerfile.*
│       └── honesty-and-scope.md   # no paths — always loaded
│   └── skills/
│       ├── dfm-domain-knowledge/SKILL.md
│       ├── occ-pythonocc-reference/SKILL.md
│       ├── research-paper-fidelity/SKILL.md
│       ├── pipeline-data-flow/SKILL.md
│       ├── evidence-and-validation/SKILL.md
│       └── run-dfm-stack/SKILL.md
└── agents/

```

```

| | |—— geometry-module-dev.md
| | |—— test-runner.md
| | |—— docs-reconciler.md
| | |—— demo-evidence-collector.md
| |—— commands/
| | |—— test.md
| | |—— debug.md
| | |—— audit.md
| | |—— status-check.md
| |—— memory/
| |—— decisions.md
| |—— known-gaps.md

```

Component-by-component detail

1. Root CLAUDE.md (~150–180 lines)

- One-paragraph identity: STEP-native DfM Agent, Level 1 complete end-to-end, Level 2 partial.
- Run commands: Docker one-liner, micromamba two-terminal flow, ports (8000/8501).
- Architecture in ~6 lines: frontend/backend split, the module chain, **stateless backend** (re-parses STEP on every single endpoint call — the single most-repeated "gotcha" across all your docs).
- Pipeline order with the one sentence that explains *why* it's ordered this way: pull direction is foundational, everything else is relative to it.
- The mutate flag contract (analyze_draft/detect_undercuts — mutate=False for scoring loops, mutate=True for the final displayed result). This shows up as a "critical" test case in testing_plan.md and a "biggest mistake" in onboarding.md, so it earns a place in the always-loaded file.
- Hard invariants: OCC only via conda/micromamba, never pip; never touch data/parts/*.stp; occ_face/occ_edge/occ_shape never get serialized directly, always .to_dict().
- The honesty rule (short version) + pointer to the full rule file and known-gaps.md.
- @ pointers to rules/skills/memory rather than inlining their content (imports still load in full at launch, so pointers here are just navigation, not @-imports of the huge docs).

2. .claude/rules/ — path-scoped, only enter context when relevant

File	Loads when touching	Core content
------	---------------------	--------------

geometry-engine.md	backend/geometry/**, backend/models/**	PartGeometry single-object pattern, face_id stability, adjacency invariants, draft formula, Boolean pruning rationale
api-layer.md	backend/api/**	endpoint list, structured error schema (code/message/operation/recovery_hint), path-traversal guard
frontend.md	frontend/**	Streamlit rerun model, session_state keys, macOS-Plotly vs. Linux-PyVista branch, "no OCC imports here" rule
testing.md	tests/**	layered order from testing_plan.md (don't skip layers), unit vs. integration markers
config-and-infra.md	config.yaml, docker-compose.yml, environment.yml, Dockerfile.*	where thresholds live, why conda not pip
honesty-and-scope.md	always loaded	condensed authority table + the exact "claims to avoid" list from DEMO_SCRIPT.md

3. .claude/skills/ — on demand, can bundle files/scripts

Skill	Triggers on	Distilled from
dfm-domain-knowledge	domain questions (draft/undercut/parting line/core-cavity)	understanding.md Parts 1–2
occ-pythonocc-reference	writing/reviewing geometry code	OCC class glossary in onboarding.md Section 3
research-paper-fidelity	"does this match the paper," report writing	Engine.md + IMPLEMENTATION_STATUS.md fidelity tables

pipeline-data-flow	tracing a field across modules, adding a module	guide.md Part B / onboarding.md Section 7
evidence-and-validation	demo prep, "give me metrics"	EVIDENCE_CHECKLIST.md + validation/perf harness usage
run-dfm-stack	booting the stack	records the exact Docker/micromamba recipe once (à la Claude Code's own / run-skill-generator pattern) so it's never re-derived

4. .claude/agents/ — isolated context, for anything verbose

Subagent	Tools	Model	Why
geometry-module-dev	Read/Grep/Glob/ Edit/Write/ Bash(pytest)	inherit	preloads occ-pythonocc-reference + pipeline-data-flow skills
test-runner	Bash(pytest*), Read, Grep	haiku	runs the layered suite, greps to failures only, returns a summary — the single biggest token saver given how verbose -v -s output is across 150+ tests
docs-reconciler	Read/Grep/Glob (read-only)	haiku	checks claims like "agent/ is empty" or "core_cavity.py is 139 lines" against actual source, operationalizing the authority table

demo-evidence-collector	Bash(validation/perf/docker), Read, Write(reports/**)	inherit	runs harnesses, fills EVIDENCE_CHECKLIST.md items, keeps Docker/CLI noise out of the main thread
-------------------------	---	---------	--

5. **.claude/commands/** — thin, kept for a few muscle-memory shortcuts

test.md, debug.md, audit.md, status-check.md — each just a one-liner that delegates to the matching subagent above with the right framing.

6. **.claude/memory/** — manually curated, git-committed (distinct from real auto-memory)

- decisions.md — append-only architecture decision log (mutate-flag design, stateless backend tradeoff, Boolean pruning gate, triangulation isolation).
- known-gaps.md — the single current "not implemented" list (agent/empty, core_cavity face-only, no PDF export, Part2.stp untested, parting line is foundation-level). CLAUDE.md will explain this is *not* the same as Claude's built-in per-machine auto-memory at ~/.claude/projects/.../memory/.

7. **.claude/settings.json + settings.local.json**

Allow (safe, read-only, matches your actual documented commands):

Bash(pytest *), Bash(python -m backend.validation.part_validation *), Bash(python -m backend.validation.performance_profile *), Bash(git status), Bash(git diff *), Bash(docker compose ps), Bash(curl http://localhost:8000/health).

Deny: Edit/Write(data/parts/**) (protect the fixture STEP files), Bash(pip install pythonocc-core*) (mirrors step_loader.py's own warning), Bash(rm -rf *).

Hooks:

- PostToolUse on Bash matching pytest → filter output to FAIL/ERROR lines only (this is Anthropic's own documented pattern, directly applicable here).
- PreToolUse on Bash matching pip install → block with a message pointing back to conda/micromamba.
- PreToolUse on Edit|Write matching data/parts/*.stp → hard block.

Open questions before I build this

1. Confirm the authority table — especially treating README.md/onboarding.md as superseded. Tell me if either has since been updated and is actually current.
2. Model choice: Haiku for test-runner/docs-reconciler okay, or Sonnet everywhere?
3. Keep .claude/commands/ at all, or fold those four into skills entirely (more modern, less visually parallel to your reference screenshot)?
4. Hooks: enable the hard blocks (pip install, data/parts/*.stp) by default,

or start softer (rules only) for now?

Let me know your answers (or just say "go with your defaults") and I'll build the whole folder.